ROS Report

Dhruv Sunil
Department of Computer Science and Engineering
Manipal Institute of Technology, Manipal, India
dhruvsunil2708@gmail.com

Abstract*—This report summarises the main communication concepts of ROS 2 Humble: the computation graph, nodes, topics, services, actions and parameters. It explains the difference between a topic and a service, how custom interfaces are created, how a colcon workspace is organised and what QoS settings do. Two problems are then solved. In Q1, a custom message carries a wheel's angular velocity and radius from a publisher to a subscriber that computes the longitudinal speed. In Q2, three nodes work together to check whether a string is a palindrome, using a custom message and a custom service. The ROS graph of each solution is shown.

***Index Terms*—ROS 2 Humble, nodes, topics, services, custom messages, colcon, QoS, rqt_graph.**

I. INTRODUCTION

The Robot Operating System 2 (ROS 2) is a set of software libraries for building robot applications as many small programs that talk to each other. By breaking complex robotics tasks into these modular pieces, called nodes, developers can easily scale systems from simple hobby projects to massive, industrial-grade multi-robot fleets.

II. THE ROS 2 COMPUTATION GRAPH

The *computation graph* is the network of ROS 2 programs that are running and exchanging data. Each program is a *node*, and nodes are connected by topics, services and actions (Fig. 1). The graph can be viewed with `rqt_graph` or listed with `ros2 node list` and `ros2 topic list`. ROS 2 has no central master; nodes find each other automatically through the DDS middleware.

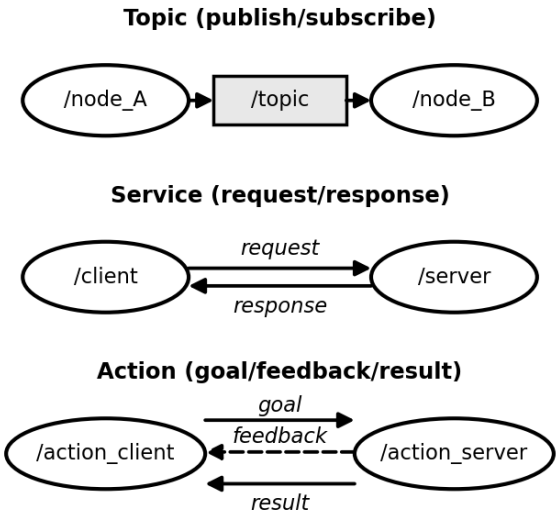


Fig. 1. The three communication types in the ROS 2 graph. Ellipses are nodes; the rectangle is a topic.

A. Nodes

A node is a single program that does one job, for example reading a sensor or checking a string. Nodes are written with a client library: `rclpy` (Python) or `rclcpp` (C++). They are started with `ros2 run <package> <executable>`.

B. Topics

A topic is a named channel for continuous data. One or more nodes publish messages to it and any number of nodes subscribe to it. The publisher does not know who is listening. Each topic carries one message type, such as `std_msgs/msg/String`.

C. Services

A service works like a function call between nodes. A *client* sends one request and the *server* sends back one response. Nothing is sent unless the client asks.

D. Actions

An action is used for tasks that take a long time, such as driving to a goal. The client sends a goal, receives regular feedback while the task runs, and gets a final result. The goal can also be cancelled. Actions are built on top of topics and services.

E. Parameters

Parameters are settings that belong to a node, such as a speed limit. They have default values, can be changed when the node starts or while it runs (`ros2 param set`), and can be listed with `ros2 param list`.

III. TOPIC VERSUS SERVICE

TABLE I
TOPIC COMPARED WITH SERVICE

Aspect	Topic	Service
Pattern	Publish / subscribe	Request / response
Who starts it	Publisher, whenever it has data	Client, when it needs something
Reply	No reply	Exactly one response
Receivers	Many subscribers	One server answers
Typical data	Continuous stream	One-off question or command
Interface file	`.msg`	`.srv` (request `---` response)

Topics should be used whenever we need a continuous stream of data while service can be used when a single response or a quick confirmation is required.

IV. CUSTOM INTERFACES

When the standard types do not fit, you can define your own. A message is described in a `.msg` file and a service in a `.srv` file, where the request and response are separated by –

--. Custom interfaces are placed in their own `ament_cmake` package, in `msg/` and `srv/` folders, and listed in `CMakeLists.txt` with `rosidl_generate_interfaces()`. The `package.xml` needs `rosidl_default_generators`, `rosidl_default_runtime` and the `rosidl_interface_packages` group.

V. COLCON AND WORKSPACE STRUCTURE

Colcon is the tool that builds all packages in a workspace. A workspace has a `src/` folder for source packages, and colcon creates `build/`, `install/` and `log/` when it builds. Python packages use `ament_python` (with `setup.py`) and interface packages use `ament_cmake`.

build: This is where the intermediate compilation files are stored.

install: This is the execution directory that contains the final, usable binaries, scripts, libraries, shared resources, and environment configuration files.

log: This directory holds the execution details, text logs, and diagnostic information for every build session. Each time you run colcon build, a new timestamped subfolder is created containing stdout, stderr, and build metrics. It is highly useful for debugging syntax or linker errors when a specific package fails to compile.

```
ros2_ws/
  src/
    taskone_interfaces/   (msg)
    taskone_py/           (Python nodes)
    rt2_interfaces/       (msg + srv)
    rt2_nodes/            (Python nodes)
  build/  install/  log/    (generated)
```

The last command must be run in every new terminal so that ROS 2 can find the built packages.

VI. QOS SETTINGS

ROS 2 leverages DDS Quality of Service (QoS) policies to fine-tune network communication across lossy wireless channels or high-throughput wired buses.

QoS Policy	Options	Architectural Impacts
Reliability	Reliable vs. Best Effort	Reliable guarantees delivery via retransmission while Best Effort drops missed packets to minimize latency (e.g., video streaming).
Durability	Transient Local vs. Volatile	Transient Local saves latest message for late-joining subscribers (e.g., map data); Volatile discards history immediately.
History	Keep Last vs. Keep All	Keep Last bounds queue size to depth N; Keep All stores all messages subject to system memory caps.
Liveliness and Depth	Liveliness vs. Depth	Deadline sets an expected time interval for data to arrive. If data doesn't arrive in time, ROS 2 triggers a warning event while Liveliness monitors whether nodes are still alive or if they crashed/disconnected from the network. .

VII. SOLUTION TO Q1

A. Problem and Approach

A custom message `TaskOne` holds two floating point values. A publisher reads the angular velocity and wheel radius typed by the user and publishes them on `/taskone`. A subscriber receives them and prints the longitudinal speed of the wheel, given by

$$v = \omega \times r \qquad (1)$$

where ω is the angular velocity in rad/s, r is the radius in m and v is the speed in m/s. Two packages were created: `taskone_interfaces` (message) and `taskone_py` (nodes).

B. Message and Code

```
# taskone_interfaces/msg/TaskOne.msg
float64 ang_vel
float64 radius
```

The publisher asks for the two values in a loop and sends each pair:

```
ang_vel = float(input('Enter angular velocity: '))
radius = float(input('Enter wheel radius: '))
msg = TaskOne()
msg.ang_vel = ang_vel
msg.radius = radius
node.publisher_.publish(msg)
```

The subscriber callback applies (1) and prints the result:

```
def listener_callback(self, msg):
    speed = msg.ang_vel * msg.radius
    self.get_logger().info(
        f'speed = {speed:.3f} m/s')
```

C. Running and ROS Graph

The subscriber is started in one terminal and the publisher in another, then `rqt_graph` is opened in a third (Fig. 2). For example, entering ω = 10 rad/s and r = 0.25 m should make the subscriber print a speed of 2.500 m/s.

```
ros2 run taskone_py taskone_subscriber
ros2 run taskone_py taskone_publisher
rqt_graph
```

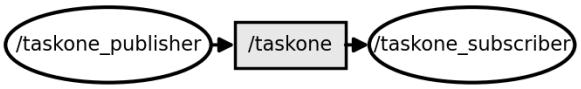


Fig. 2. ROS graph for Q1: publisher, topic /taskone and subscriber.

VIII. SOLUTION TO Q2

A. Problem and Approach

Three nodes are needed. Node 1 publishes strings typed by the user. Node 2 checks if each string is a palindrome and sends 1 (true) or 0 (false) to Node 3. Node 3 answers "Yes" for 1 and "No" for 0, and Node 2 prints the answer. The packages are `rt2_interfaces` (messages and service) and `rt2_nodes` (the three nodes).

Node 1 to Node 2 is a continuous flow of strings, so a **topic** `/rt2_string` with the custom message `RT2String` is used. Node 2 to Node 3 is a question that needs an answer back, so a **service** `/check_palindrome` is used, with Node 2 as the client and Node 3 as the server. This follows the rule from Section III.

B. Interfaces

```
# rt2_interfaces/msg/RT2String.msg
string data

# rt2_interfaces/srv/PalindromeCheck.srv
int32 is_palindrome
---
string answer
```

C. Node Code

Node 1 reads a string and publishes it, repeating until stopped:

```
text = input('Enter a string: ').strip()
msg = RT2String()
msg.data = text
node.publisher_.publish(msg)
```

Node 2 checks the string and calls the service:

```
def string_callback(self, msg):
    text = msg.data
    flag = 1 if text == text[::-1] else 0
    request = PalindromeCheck.Request()
    request.is_palindrome = flag
    future = self.client.call_async(request)
    future.add_done_callback(
        partial(self.response_callback,
                text, flag))
```

Node 3 answers the request:

```
def handle_request(self, request, response):
    if request.is_palindrome == 1:
        response.answer = 'Yes'
    else:
        response.answer = 'No'
    return response
```

D. Running and ROS Graph

The nodes are started in three terminals (Node 3 first, so the service exists when Node 2 starts), and `rqt_graph` is opened in a fourth (Fig. 3). Typing `AABAA` should print "Yes" in Node 2's terminal and typing `AB` should print "No".

```
ros2 run rt2_nodes node3_responder
ros2 run rt2_nodes node2_checker
ros2 run rt2_nodes node1_publisher
rqt_graph
```

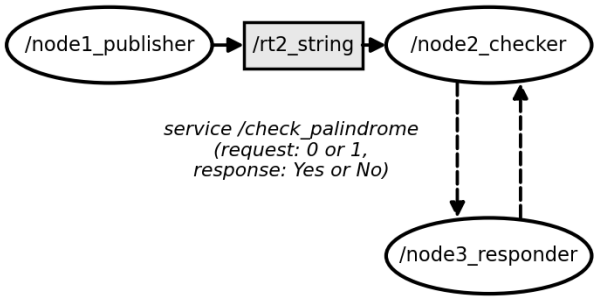


Fig. 3. ROS graph for Q2. The dashed link is the service, which rqt_graph itself does not draw.

`rqt_graph` only draws topics, so the service link between Node 2 and Node 3 does not appear in the real screenshot. It can be confirmed with `ros2 service list` and `ros2 node info /node2_checker`.

IX. CONCLUSION

Nodes, topics, services, actions and parameters form the ROS 2 computation graph. Topics suit continuous data and services suit request/response exchanges. Custom messages and services extend these to application data, and colcon builds them in a workspace.

REFERENCES

[1] Open Robotics, "ROS 2 Documentation: Humble," docs.ros.org.